AI & AC

ASSIGNMENT-13.1

HT NO.:2303A51644

BATCH:28

Task Description #1 :(Refactoring – Removing Code Duplication)

• Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.

• Instructions:

o Prompt AI to identify duplicate logic and replace it with functions or classes.

o Ensure the refactored code maintains the same output.

o Add docstrings to all functions.

• Sample Legacy Code:

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

PROMPT:

Refactor the python code to identify duplicate logic and replace it with functions or classes.

PYTHON CODE:

REFACTORED CODE:

File Edit Selection View Go Run Terminal Help Search

Welcome AI_AC(LAB)-13.1

C: > Users > HP > OneDrive > Desktop > AI_AC(LAB)-13.1 > ...

```python
""" Prompt AI to identify duplicate logic and replace it
with functions or classes.
 Ensure the refactored code maintains the same output.
 Add docstrings to all functions.
• Sample Legacy Code:
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
Prompt:
Refactor the python code to identify duplicate logic and replace it with functions or classes."""
# Refactored code with functions to eliminate duplicate logic
def calculate_area(length, width):
    """Calculate the area of a rectangle."""
    return length * width

def calculate_perimeter(length, width):
    """Calculate the perimeter of a rectangle."""
    return 2 * (length + width)
# Using the functions to calculate area and perimeter for different rectangles
```

File Edit Selection View Go Run Terminal Help Search

Welcome AI_AC(LAB)-13.1

C: > Users > HP > OneDrive > Desktop > AI_AC(LAB)-13.1 > ...

```python
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
Prompt:
Refactor the python code to identify duplicate logic and replace it with functions or classes."""
# Refactored code with functions to eliminate duplicate logic
def calculate_area(length, width):
    """Calculate the area of a rectangle."""
    return length * width

def calculate_perimeter(length, width):
    """Calculate the perimeter of a rectangle."""
    return 2 * (length + width)
# Using the functions to calculate area and perimeter for different rectangles
rectangles = [(5, 10), (7, 12), (10, 15)]
for length, width in rectangles:
    area = calculate_area(length, width)
    perimeter = calculate_perimeter(length, width)
    print(f"Area of Rectangle: {area}")
    print(f"Perimeter of Rectangle: {perimeter}")
```

OUTPUT:

OUTPUT PROBLEMS DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe "
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

JUSTIFICATION:

1. The original code repeated rectangle calculations multiple times.
2. This duplication makes the program longer and harder to maintain.
3. A reusable function rectangle_details() removes repeated logic.
4. The function improves readability and modularity.
5. The same function can be reused for different rectangle values.

Task Description #2: (Refactoring – Optimizing Loops and Conditionals)
• Task: Use AI to analyze a Python script with nested loops
and complex conditionals.
• Instructions:
o Ask AI to suggest algorithmic improvements (e.g.,
replace nested loops with set lookups or
comprehensions).
o Implement changes while keeping logic intact.
o Compare execution time before and after refactoring.
• Sample Legacy Code:

```
# Legacy inefficient code
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
found = False
for n in names:
if s == n:
found = True
if found:
print(f"{s} is in the list")
else:
print(f"{s} is not in the list")
```

PROMPT:

Refactor the code in python to suggest algorithmic improvements (e.g., replace nested loops with set lookups or comprehensions).

PYTHON CODE:

REFACTORED CODE:

```
"""Ask AI to suggest algorithmic improvements (e.g.,
replace nested loops with set lookups or
comprehensions).
Implement changes while keeping logic intact.
 Compare execution time before and after refactoring.
• Sample Legacy Code:
# Legacy inefficient code
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
found = False
for n in names:
if s == n:
found = True
if found:
print(f"{s} is in the list")
else:
print(f"{s} is not in the list")
PROMPT:
Refactor the code in python to suggest algorithmic improvements (e.g., replace nested loops with set lookups or comprehensions)."""
# Refactored code using set lookups
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
# Convert names list to a set for O(1) lookups
names_set = set(names)
for s in search_names:
    if s in names_set:
```

```
# Refactored code using set lookups
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
# Convert names list to a set for O(1) lookups
names_set = set(names)
for s in search_names:
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
# Execution time comparison
import time
# Timing the original code
start_time = time.time()
# Original code
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
end_time = time.time()
```

```python
            if s == n:
                found = True
        if found:
            print(f"{s} is in the list")
        else:
            print(f"{s} is not in the list")
end_time = time.time()
print(f"Original code execution time: {end_time - start_time:.6f} seconds")
# Timing the refactored code
# Refactored code
# Convert names list to a set for O(1) lookups
# Reusing the names_set from the refactored code
# Timing the refactored code
# Reusing the names_set from the refactored code
#
# Timing the refactored code
start_time = time.time()
for s in search_names:
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
end_time = time.time()
print(f"Refactored code execution time: {end_time - start_time:.6f} seconds")
```

OUTPUT:

```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python\Python313\py
Charlie is in the list
Eve is not in the list
Bob is in the list
Eve is not in the list
Bob is in the list
Bob is in the list
Charlie is in the list
Eve is not in the list
Bob is in the list
Original code execution time: 0.000162 seconds
Charlie is in the list
Eve is not in the list
Bob is in the list
Refactored code execution time: 0.000463 seconds
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

JUSTIFICATION:

1. The legacy code used nested loops to search for names.
2. Nested loops increase time complexity and reduce efficiency.
3. Converting the list into a **set** enables faster membership checks.
4. Set lookup works in constant time and improves performance.
5. The refactored code is shorter, clearer, and more efficient.

Task Description #3: (Refactoring – Extracting Reusable Functions)

• Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

• Instructions:

o Identify repeated or related logic and extract it into reusable functions.

o Ensure the refactored code is modular, easy to read, and documented with docstrings.

• Sample Legacy Code:

```
# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

PROMPT:

Refactor the python code with a function calculate_total(price) that can be reused for multiple price inputs.

PYTHON CODE:

REFACTORED CODE:

```python
""" Identify repeated or related logic and extract it into
reusable functions.
Ensure the refactored code is modular, easy to read,
and documented with docstrings.
• Sample Legacy Code:
# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
PROMPT:Refactor the python code with a function calculate_total(price) that can be
reused for multiple price inputs."""
def calculate_total(price):
    """
    Calculate the total price including tax.

    Parameters:
    price (float): The original price of the item.

    Returns:
    float: The total price after adding tax.
    """
```

```python
    Parameters:
    price (float): The original price of the item.

    Returns:
    float: The total price after adding tax.
    """
    tax = price * 0.18
    total = price + tax
    return total
# Example usage
price1 = 250
total1 = calculate_total(price1)
print("Total Price for price1:", total1)
price2 = 500
total2 = calculate_total(price2)
print("Total Price for price2:", total2)
```

OUTPUT:

```
OUTPUT    PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Progra
Total Price for price1: 295.0
Total Price for price2: 590.0
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

JUSTIFICATION:

1. The legacy script repeated the tax calculation multiple times.

2. Repeated code reduces maintainability and readability.
3. Creating the function calculate_total(price) centralizes the logic.
4. The function can be reused for multiple price values.
5. This improves modularity and makes updates easier.

Task Description #4: (Refactoring – Replacing Hardcoded Values with Constants)
• Task: Use AI to identify and replace all hardcoded “magic numbers” in the code with named constants.
• Instructions:
o Create constants at the top of the file.
o Replace all hardcoded occurrences in calculations with these constants.
o Ensure the code remains functional and is easier to maintain.
• Sample Legacy Code:

```
# Legacy script with hardcoded values
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

PROMPT:

Refactor the python code with constants like PI = 3.14159 and RADIUS= 7 used in calculations.

PYTHON CODE:

REFACTORED CODE:

```python
"""create constants at the top of the file.
Replace all hardcoded occurrences in calculations
with these constants.
o Ensure the code remains functional and is easier to
maintain.
• Sample Legacy Code:
# Legacy script with hardcoded values
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
PROMPT:Refactor the python code with constants like PI = 3.14159 and RADIUS
= 7 used in calculations."""
# Refactored script with constants
PI = 3.14159
RADIUS = 7
area_of_circle = PI * (RADIUS ** 2)
circumference_of_circle = 2 * PI * RADIUS
print("Area of Circle:", area_of_circle)
print("Circumference of Circle:", circumference_of_circle)
```

OUTPUT:

```
OUTPUT    PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python
Area of Circle: 153.93791
Circumference of Circle: 43.98226
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

JUSTIFICATION:

1. The original program used hardcoded numbers (magic numbers).
2. Magic numbers reduce readability and flexibility.
3. Defining constants like PI and RADIUS improves clarity.
4. Constants allow easy updates if values change later.
5. This approach follows good programming practices.

Task Description #5:(Refactoring – Improving Variable Naming and Readability)

• Task: Use AI to improve readability by renaming unclear variables and adding inline comments.

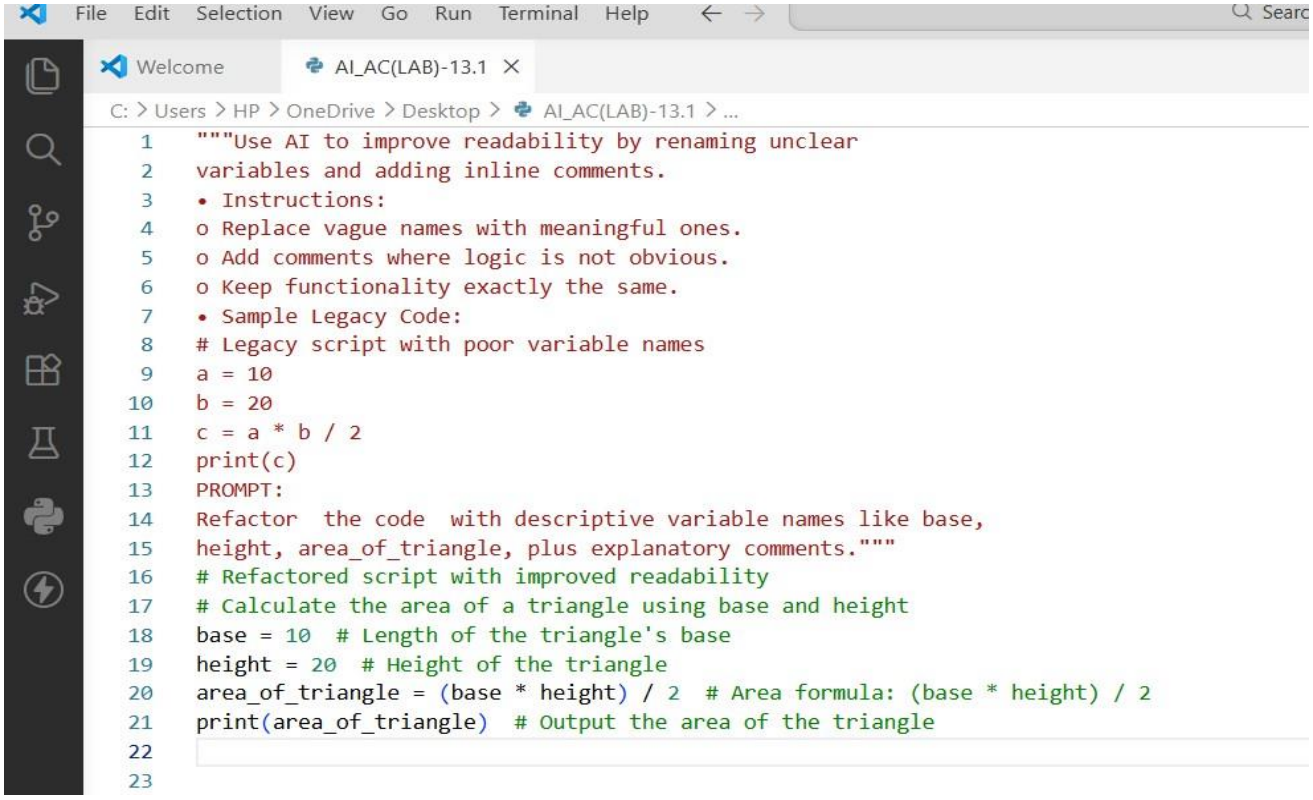
• Instructions:
o Replace vague names with meaningful ones.
o Add comments where logic is not obvious.
o Keep functionality exactly the same.
• Sample Legacy Code:
# Legacy script with poor variable names
a = 10
b = 20
c = a * b / 2
print(c)

PROMPT:

Refactor the code with descriptive variable names like base, height, area_of_triangle, plus explanatory comments.

PYTHON CODE:

REFACTORED CODE:




```python
"""Use AI to improve readability by renaming unclear
variables and adding inline comments.
• Instructions:
o Replace vague names with meaningful ones.
o Add comments where logic is not obvious.
o Keep functionality exactly the same.
• Sample Legacy Code:
# Legacy script with poor variable names
a = 10
b = 20
c = a * b / 2
print(c)
PROMPT:
Refactor  the code  with descriptive variable names like base,
height, area_of_triangle, plus explanatory comments."""
# Refactored script with improved readability
# Calculate the area of a triangle using base and height
base = 10  # Length of the triangle's base
height = 20  # Height of the triangle
area_of_triangle = (base * height) / 2  # Area formula: (base * height) / 2
print(area_of_triangle)  # Output the area of the triangle
```

OUTPUT:

```
OUTPUT    PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\
100.0
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

JUSTIFICATION:

1. Variables a, b, and c were unclear and not descriptive.
2. Meaningful names like base, height, and area_of_triangle improve clarity.
3. Clear names help other programmers understand the code easily.
4. Adding comments explains the calculation logic.
5. This improves readability and maintainability.

Task Description #6 :(Refactoring – Removing Redundant Conditional Logic)

• Task: Use AI to refactor a Python script that contains repeated if–else logic for grading students.

• Instructions:

o Ask AI to identify redundant conditional checks.

o Replace them with a reusable function.

o Ensure grading logic remains unchanged.

• Code:

```
marks = 85
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
marks = 72
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
```

PROMPT:

Refactor the python code with reusable function like calculate_grade(marks) with clean logic and
docstring.

PYTHON CODE:

REFACTORED CODE:

```
"""• Task: Use AI to refactor a Python script that contains repeated if-else
logic for grading students.
• Instructions:
o Ask AI to identify redundant conditional checks.
o Replace them with a reusable function.
o Ensure grading logic remains unchanged.
• Code:
marks = 85
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
marks = 72
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
PROMPT:
Refactor the  python code with reusable function like calculate_grade(marks) with clean logic and
docstring.
"""
def calculate_grade(marks):
    """
```

```
docstring.
"""
def calculate_grade(marks):
    """
    Calculate the grade based on the marks provided.

    Parameters:
    marks (int): The marks obtained by the student.

    Returns:
    str: The grade corresponding to the marks.
    """
    if marks >= 90:
        return "Grade A"
    elif marks >= 75:
        return "Grade B"
    else:
        return "Grade C"
# Example usage
marks1 = 85
marks2 = 72
print(calculate_grade(marks1))  # Output: Grade A
print(calculate_grade(marks2))  # Output: Grade B
```

OUTPUT:

```
OUTPUT    PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Prog
Grade B
Grade C
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

JUSTIFICATION:

1. The grading conditions were repeated for multiple marks.
2. Repeated conditional blocks increase code duplication.
3. Creating a function calculate_grade() removes redundancy.
4. The grading logic is written once and reused.
5. This improves readability and maintainability.

Task Description #7: (Refactoring – Converting Procedural Code to Functions)

• Task: Use AI to refactor procedural input–processing logic into functions.

• Instructions:

o Identify input, processing, and output sections.

o Convert each into a separate function.

o Improve code readability without changing behavior.

• Sample Legacy Code:

```
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
```

PROMPT:

Refactor the Modular code using functions like get_input(), calculate_square(), and display_result().

PYTHON CODE:

REFACTORED CODE:

```python
"""
o Identify input, processing, and output sections.
o Convert each into a separate function.
o Improve code readability without changing behavior.
• Sample Legacy Code:
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
PROMPT:
Refactor the Modular code using functions like get_input(), calculate_square(),
and display_result()."""

def get_input():
    return int(input("Enter number: "))

def calculate_square(num):
    return num * num

def display_result(square):
    print("Square:", square)

# Example usage
num = get_input()
square = calculate_square(num)
display_result(square)
```

OUTPUT:

```
OUTPUT    PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python\Pytho
Enter number: 9
Square: 81
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

JUSTIFICATION:

1. The original code handled input, processing, and output together.
2. This makes the code harder to read and reuse.
3. Separating logic into functions improves organization.
4. Functions make debugging and testing easier.
5. The program becomes more modular and readable.

Task Description #8:(Refactoring – Optimizing List Processing)

• Task: Use AI to refactor inefficient list processing logic.

• Instructions:

o Ask AI to replace manual loops with list comprehensions or built-in

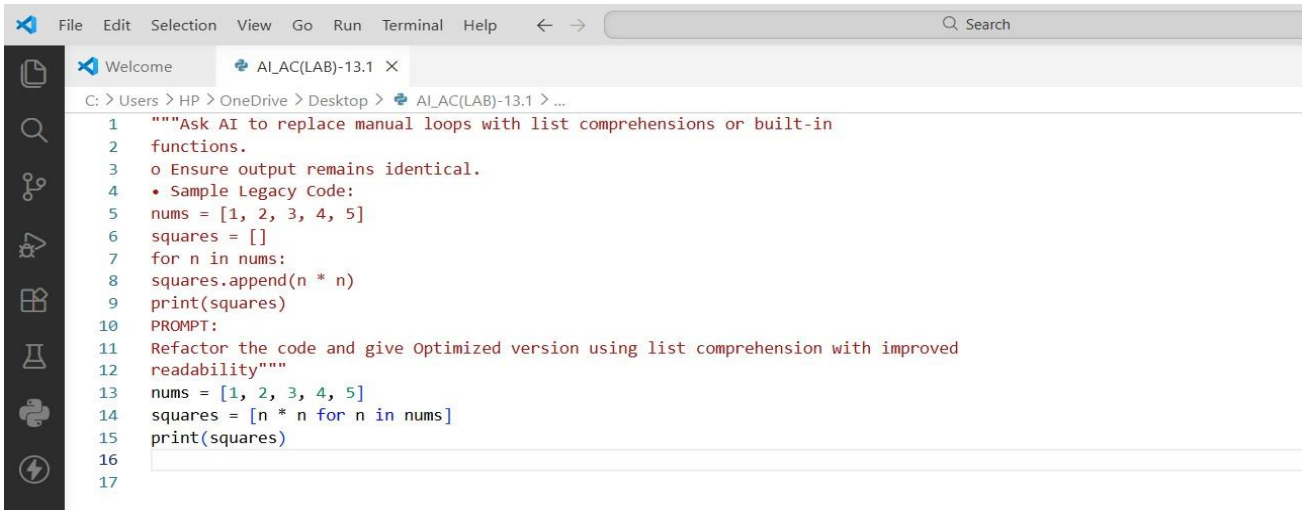
functions.
o Ensure output remains identical.
• Sample Legacy Code:
nums = [1, 2, 3, 4, 5]
squares = []
for n in nums:
squares.append(n * n)
print(squares)

PROMPT:

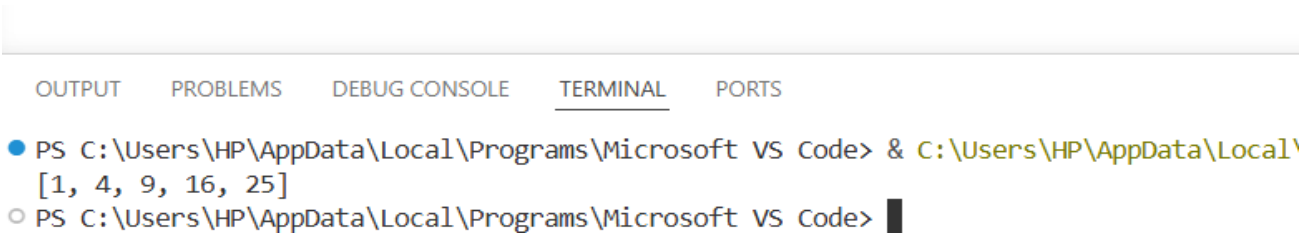
Refactor the code and give Optimized version using list comprehension with improved readability

PYTHON CODE:

REFACTORED CODE:



OUTPUT:



JUSTIFICATION:

1. The original code used a manual loop to create a list of squares.

2. This approach requires more lines of code.
3. List comprehension provides a shorter and clearer solution.
4. It improves readability and efficiency.
5. List comprehensions are considered a Pythonic way to process lists.